



Tartalomjegyzék

1.Specifikáció.....	1
1.1. A feladat szöveges leírása	1
1.2. Követelmények részletezése	2
2. Tervezés.....	2
2.1. Architektúra és technológiai stack	2
2.2. Konténerizációs és orkesztrációs terv	2
2.3. Skálázási és erőforrás-kezelési terv (HPA).....	3
3. Megvalósítás.....	3
3.1. A .NET Minimal API backend és statikus frontend megvalósítása	3
3.2. Multi-stage Dockerfile elkészítése	3
3.3. Kubernetes manifestek (Deployment, Service, HPA) konfigurálása	4
4. Tesztelés	5
4.1. Helyi konténerizált futtatás (Docker Desktop)	5
4.2. Kubernetes orkesztráció és HPA terhelésteszt (Minikube)	6
Összegzés.....	7

1. Specifikáció

1.1. A feladat szöveges leírása

A projekt célja egy konténerizált microservice architektúra megvalósítása és vizsgálata Kubernetes környezetben. Az alkalmazás egy beépített CPU-terhelő modul segítségével demonstrálja az erőforrás-kihasználtságon alapuló automatikus horizontális skálázódást (Horizontal Pod Autoscaler - HPA).

A rendszer két fő elemből áll: egy könnyűsúlyú ASP.NET Core 8 Minimal API backendből, amely dinamikus HTML/JS dashboardot szolgál ki, valamint a Kubernetes orkesztrációs rétegből, amely gondoskodik a podok automatikus skálázásáról és a terheléselosztásról.



1.2. Követelmények részletezése

Szerveroldali követelmények: .NET 8 runtime, REST API végpontok biztosítása a konténer metaadatainak lekéréséhez, valamint szabályozható időtartamú CPU-terhelés generálásához.

Kliensoldali követelmények: Tisztán HTML/JavaScript alapú, böngészőből elérhető dashboard, amely interaktív módon jeleníti meg a válaszoló konténer azonosítóját, és gombnyomásra indítja a terheléstesztet.

Konténerizációs követelmények: Multi-stage Dockerfile használata az image méretének és a biztonsági kockázatoknak a csökkentésére.

Orkesztrációs követelmények: Kubernetes Deployment, és Metrics-Server alapú HPA konfiguráció a CPU-terheltség alapján történő dinamikus pod-skálázáshoz (min. 1, max. 5 pod).

2. Tervezés

2.1. Architektúra és technológiai stack

- Backend: C# / ASP.NET Core 8 Minimal API.
- Frontend: Statikus HTML5 / JavaScript (Fetch API segítségével kommunikál a REST API-val).
- Konténerizáció: Docker (Docker Desktop Engine).
- Orkesztráció: Kubernetes (Minikube helyi fejlesztői cluster).

2.2. Konténerizációs és orkesztrációs terv

Multi-stage Docker build tervezése:

- Build szakasz:

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
```

- Runtime szakasz:

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
```



Kubernetes orkesztrációs architektúra tervezése:

- Deployment: Definiálja a podok elvárt állapotát, a replikák alapértelmezett számát, valamint az erőforrás-korlátokat, amelyek elengedhetetlenek a méréshez.
- Service: Biztosítja a belső és külső hálózati hozzáférést, valamint az automatikus terheléselosztást a futó pod példányok között.

2.3. Skálázási és erőforrás-kezelési terv (HPA)

A Kubernetes HPA a metrics-server által gyűjtött CPU-értékek alapján hoz döntést.

- Pod CPU limitek: A podok számára meghatározott CPU kérés és korlát került beállításra (pl. requests.cpu: 100m, limits.cpu: 500m).
- Autoscaling küszöb: A HPA célja, hogy a podok átlagos CPU-kihasználtságát 50% alatt tartsa. Amint a terhelés ezt meghaladja, új pod példányok indulnak el.

3. Megvalósítás

3.1. A .NET Minimal API backend és statikus frontend megvalósítása

A backendben a UseDefaultFiles() és UseStaticFiles() middleware-ek sorrendje precízen került beállításra a statikus gyökérfájlok kiszolgálásához. A főbb végpontok:

- GET /api/info: Visszaadja a konténer egyedi azonosítóját (Environment.MachineName) és a serveridőt.
- POST /api/stress?durationSeconds={sec}: CPU-intenzív matematikai ciklust futtat a megadott másodpercig.

3.2. Multi-stage Dockerfile elkészítése

A Dockerfile felépítése biztosítja a minimális image méretet:

```
1 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
2 WORKDIR /src
3
4 COPY *.csproj ./
5 RUN dotnet restore
6
7 COPY . ./
8 RUN dotnet publish -c Release -o /app/out /p:UseAppHost=false
9
10 FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
11 WORKDIR /app
12 COPY --from=build /app/out .
13
14 EXPOSE 80
15 ENV ASPNETCORE_URLS=http://+:80
16
17 ENTRYPOINT ["dotnet", "CloudProjectApplication.dll"]
```

3.3. Kubernetes manifestek (Deployment, Service, HPA) konfigurálása

A k8s/ könyvtárban található manifest fájlok leírják a Kubernetes objektumokat:

- Deployment: Meghatározza az alkalmazás podjait és az erőforrás-limiteket.

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: cloud-backend-deployment
5    labels:
6      app: cloud-backend
7  spec:
8    replicas: 3
9    selector:
10     matchLabels:
11       app: cloud-backend
12  template:
13    metadata:
14      labels:
15        app: cloud-backend
16    spec:
17      containers:
18        - name: cloud-backend-container
19          image: cloud-backend:v1
20          imagePullPolicy: Never
21          ports:
22            - containerPort: 80
23          resources:
24            requests:
25              cpu: "100m"
26              memory: "64Mi"
27            limits:
28              cpu: "500m"
29              memory: "128Mi"
```

- Service: NodePort (vagy ClusterIP) típusú szolgáltatás a podok közötti terheléelosztásra.

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: cloud-backend-service
5  spec:
6    type: NodePort
7    selector:
8      app: cloud-backend
9    ports:
10     - protocol: TCP
11       port: 80
12       targetPort: 80
13       nodePort: 30080
```

- HPA: HorizontalPodAutoscaler manifest, amely a CPU küszöbértéket figyelje.

```
1  apiVersion: autoscaling/v2
2  kind: HorizontalPodAutoscaler
3  metadata:
4    name: cloud-backend-hpa
5  spec:
6    scaleTargetRef:
7      apiVersion: apps/v1
8      kind: Deployment
9      name: cloud-backend-deployment
10   minReplicas: 1
11   maxReplicas: 5
12   metrics:
13   - type: Resource
14     resource:
15       name: cpu
16       target:
17         type: Utilization
18         averageUtilization: 50
```

4. Tesztelés

4.1. Helyi konténerizált futtatás (Docker Desktop)

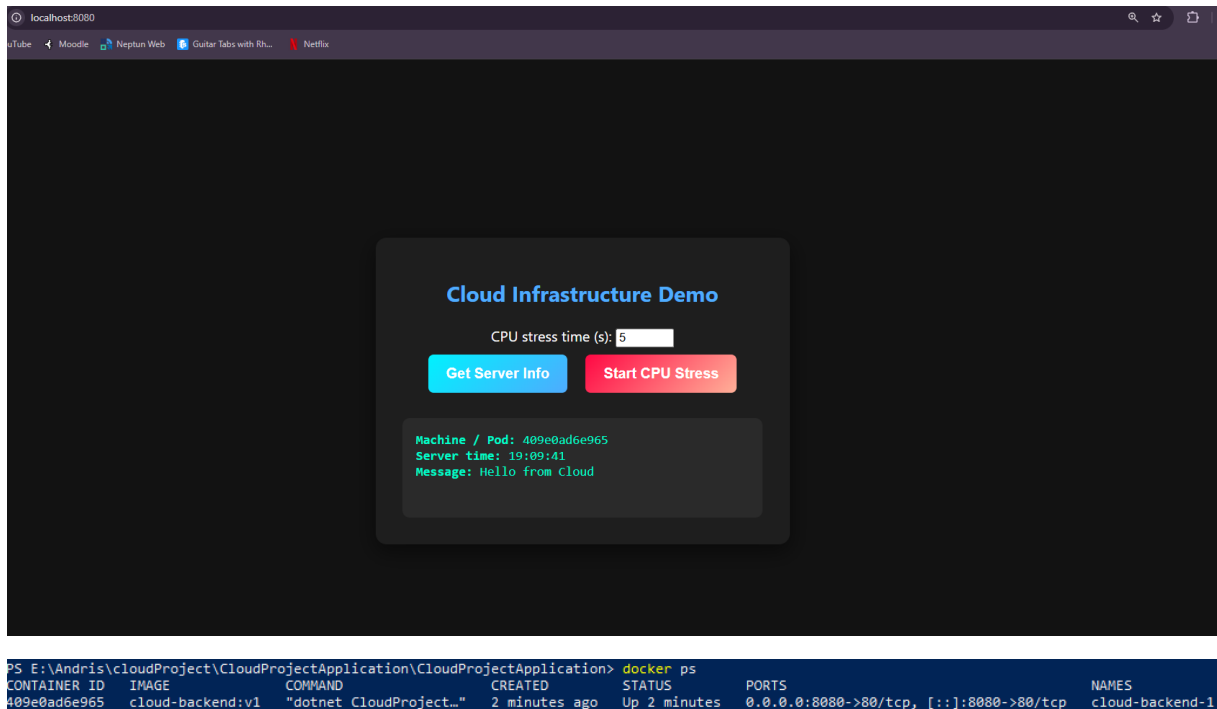
Image build:

```
docker build -t cloud-backend:v1 .
```

Konténer létrehozás:

```
docker run -d -p 8080:80 --name cloud-backend-1 cloud-backend:v1
```

Ellenőrzés:



The screenshot shows a web browser window at localhost:8080 displaying a 'Cloud Infrastructure Demo' application. The application has a dark theme and includes a form for 'CPU stress time (s):' with a value of 5. There are two buttons: 'Get Server Info' (blue) and 'Start CPU Stress' (red). Below the buttons, a text box displays the following information: Machine / Pod: 409e0ad6e965, Server time: 19:09:41, and Message: Hello from Cloud.

Below the browser window, a terminal window shows the command `docker ps` and its output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
409e0ad6e965	cloud-backend:v1	"dotnet CloudProject..."	2 minutes ago	Up 2 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	cloud-backend-1

Valóban a megfelelő ID-val rendelkező konténer fut a 8080-as porton.

4.2. Kubernetes orkesztráció és HPA terhelésteszt (Minikube)

A Minikube cluster elindítása és az image betöltése:

```
minikube start --driver=docker
```

```
minikube image load cloud-backend:v1
```

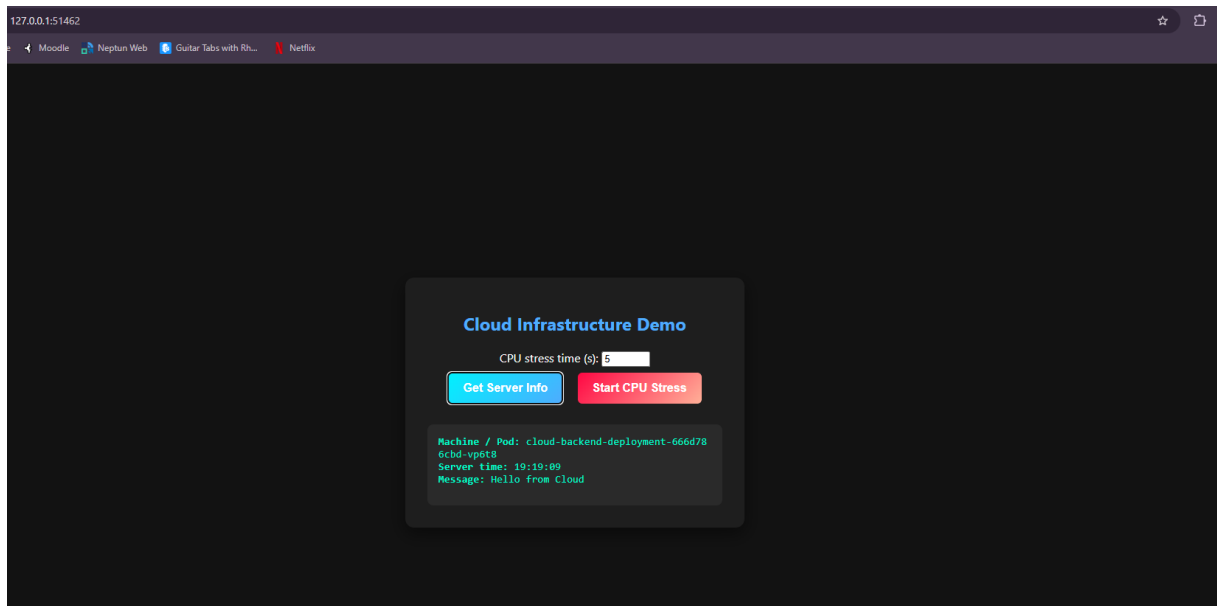
Manifestek alkalmazása és a szolgáltatás megnyitása:

```
kubectl apply -f k8s/
```

```
minikube service cloud-backend-service
```



Ellenőrzés:



Látni hogy már az imént elkészített pod-ról fut az alkalmazás.

Ezután a stressz teszthez...

Metrics-Server engedélyezése:

```
minikube addons enable metrics-server
```

Automatikus skálázódás (HPA) megfigyelése:

```
kubectl get hpa cloud-backend-hpa -w
```

Ellenőrzés:

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 1%/50%	1	5	1	6m36s
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 112%/50%	1	5	1	8m15s
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 112%/50%	1	5	3	8m30s
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 1%/50%	1	5	3	9m15s
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 1%/50%	1	5	3	14m
cloud-backend-hpa	Deployment/cloud-backend-deployment	cpu: 1%/50%	1	5	1	14m

Összegzés

A tesztek igazolták, hogy a Kubernetes infrastruktúra sikeresen érzekelte a CPU-terhelést a Metrics-Serveren keresztül, és a HPA beállításoknak megfelelően dinamikusan új podokat példányosított, majd a terhelés megszűnésével automatikusan visszaskálázta a rendszert.

Közreműködött: Gemini 3.6 Flash

Lektorálta: Deák András Botond